Python & Pandas — Complete Reference

EY Probation Training — Week 1. Roadmap scope: basics, data types, loops, functions, lists, dictionaries, pandas basics. Outcome bar: "write Python scripts and perform basic data analysis."

---

1. Data Types

Type	Mutable	Ordered	Literal	Notes
`int`, `float`, `bool`	—	—	`5`, `2.5`, `True`	`bool` is a subclass of `int`
`str`	No	Yes	`"text"`	Immutable — methods return new strings
`list`	Yes	Yes	`[1, 2, 3]`	The default sequence
`tuple`	No	Yes	`(1, 2)`	Hashable, so usable as a dict key
`dict`	Yes	Yes (3.7+)	`{"a": 1}`	Keys must be hashable
`set`	Yes	No	`{1, 2}`	Unique elements, fast membership
`NoneType`	—	—	`None`	Test with `is None`, never `== None`

```
type(x)              # exact type
isinstance(x, int)   # preferred check — respects inheritance
int("42"); float("3.14"); str(42); list("abc"); bool(0)   # conversions
```

2. Strings

```
s = "  Hello, World  "
s.strip(); s.lstrip(); s.rstrip()
s.lower(); s.upper(); s.title()
s.replace("World", "Python")
s.split(",")                 # -> ['  Hello', ' World  ']
",".join(["a", "b", "c"])    # -> 'a,b,c'
s.startswith("H"); s.endswith("d"); "World" in s
s.find("World")              # -1 if absent
s.index("World")             # raises ValueError if absent
```

f-strings (use these, not `%` or `.format()`):

```
name, val = "Q3", 1234.5678
f"{name}: {val:,.2f}"        # 'Q3: 1,234.57'
f"{val:>12.1f}"              # right-align in 12 chars
f"{0.4567:.1%}"              # '45.7%'
f"{name=}"                   # 'name=Q3'  (debug form)
```

Slicing (applies to lists too):

```
s[0]; s[-1]; s[2:5]; s[:3]; s[3:]; s[::2]; s[::-1]   # last one reverses
```

3. Lists

```
xs = [3, 1, 4, 1, 5]
xs.append(9)          # add one at the end
xs.extend([2, 6])     # add many
xs.insert(0, 0)       # at an index
xs.remove(1)          # first matching VALUE
xs.pop()              # last item (and returns it)
xs.pop(0)             # by index
xs.sort()             # IN PLACE, returns None
sorted(xs)            # returns a NEW list
xs.sort(reverse=True)
xs.sort(key=len)      # sort by a computed key
xs.reverse(); xs.count(1); xs.index(4)
len(xs); sum(xs); min(xs); max(xs)
```

Common trap: `xs.sort()` returns `None`. Writing `xs = xs.sort()` destroys your list. Use `sorted()` when you want a return value.

List comprehensions

```
squares  = [x**2 for x in range(10)]
evens    = [x for x in xs if x % 2 == 0]
labelled = [f"item_{x}" for x in xs]
pairs    = [(x, y) for x in [1,2] for y in "ab"]     # nested loops
cond     = [x if x > 0 else 0 for x in xs]           # ternary goes BEFORE 'for'
```

Filtering `if` goes after the `for`; a conditional *value* goes before it. Mixing these up is a classic error.

```
{k: v for k, v in pairs}        # dict comprehension
{x % 3 for x in range(10)}      # set comprehension
(x**2 for x in range(10))       # GENERATOR — lazy, not a list
```

Copying

```
a = [1, 2, 3]
b = a            # SAME object — mutating b changes a
c = a.copy()     # shallow copy (or a[:], or list(a))
import copy; d = copy.deepcopy(nested)   # for nested structures
```

4. Dictionaries

```
d = {"a": 1, "b": 2}
d["a"]              # KeyError if missing
d.get("z")          # None if missing
d.get("z", 0)       # default if missing
d["c"] = 3
d.setdefault("e", []).append(1)
d.update({"f": 6})
d.pop("a"); del d["b"]
"a" in d            # membership tests KEYS

d.keys(); d.values(); d.items()
for k, v in d.items(): print(k, v)

sorted(d.items(), key=lambda kv: kv[1], reverse=True)    # sort by value
max(d, key=d.get)                                        # key with the largest value
```

Counting pattern:

```
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1

from collections import Counter, defaultdict
Counter(words).most_common(3)
groups = defaultdict(list)
for name, dept in rows: groups[dept].append(name)
```

5. Control Flow

```python
if x > 10:      pass
elif x > 5:     pass
else:           pass

for i in range(5):              # 0..4
for i in range(2, 10, 2):       # 2,4,6,8
for item in xs:
for i, item in enumerate(xs, start=1):
for a, b in zip(xs, ys):
for k, v in d.items():

while cond:
    ...
    if done: break
    if skip: continue
else:
    ...   # runs only if the loop was NOT broken out of
```

`enumerate` and `zip` are what assessments look for instead of `for i in range(len(xs))`.

6. Functions

```python
def summarise(data, top=5, *args, label="Report", **kwargs):
    """One-line summary.

    Args:
        data: the input list.
        top: how many rows to return.
    Returns:
        list: the top-n items.
    """
    return sorted(data, reverse=True)[:top]
```

- Positional args first, then defaults, then `*args`, then keyword-only, then `**kwargs`.
- **Never use a mutable default.** `def f(x, acc=[])` shares one list across all calls. Use `acc=None` then `if acc is None: acc = []`.
- Type hints are good practice: `def f(x: int, names: list[str]) -> dict:`

```python
sq = lambda x: x**2                             # anonymous function
sorted(rows, key=lambda r: (r["dept"], -r["pay"]))   # multi-key sort
list(map(str.upper, names))
list(filter(lambda x: x > 0, xs))
```

Comprehensions are preferred over `map` / `filter` in modern Python style.

7. Files and Errors

```python
with open("data.txt") as f:          # 'with' closes the file automatically
    text = f.read()
    # f.readlines()  /  for line in f:

with open("out.txt", "w") as f:
    f.write("hello\n")
    f.writelines(lines)

import csv
with open("data.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["Region"])

import json
cfg = json.load(open("cfg.json"))
json.dump(obj, open("out.json", "w"), indent=2)
```

Modes: `r` read, `w` write (truncates), `a` append, `rb` / `wb` binary.

```python
try:
    val = int(s)
except ValueError as e:
    print(f"bad value: {e}")
except (TypeError, KeyError):
    pass
else:
    print("no exception")     # runs if try succeeded
finally:
    cleanup()                 # always runs
raise ValueError("explain what went wrong")
```

8. Pandas — Creating and Inspecting

```python
import pandas as pd, numpy as np

df = pd.read_csv("file.csv")
df = pd.read_csv("f.csv", parse_dates=["date"], dtype={"id": str},
                 usecols=["a","b"], na_values=["NA","-"], thousands=",")
df = pd.read_excel("f.xlsx", sheet_name="Orders")
df = pd.DataFrame({"a": [1,2], "b": ["x","y"]})

df.head(); df.tail(); df.sample(5)
df.shape; df.columns; df.dtypes; df.index
df.info()             # dtypes + non-null counts + memory
df.describe()         # numeric summary
df.describe(include="object")   # categorical summary
df.nunique(); df["col"].unique(); df["col"].value_counts()
df.isna().sum()       # nulls per column
df.memory_usage(deep=True)
```

9. Pandas — Selecting

```python
df["col"]                 # Series
df[["a", "b"]]            # DataFrame

# .loc = LABEL based (end-inclusive) | .iloc = POSITION based (end-exclusive)
df.loc[0, "col"]
df.loc[0:5, "a":"c"]      # includes row 5 AND column c
df.iloc[0:5, 0:3]         # excludes row 5 and column 3
df.loc[df["qty"] > 5, ["region", "qty"]]

# Boolean filtering — each condition needs parentheses
df[(df.qty > 5) & (df.region == "West")]      # & | ~   NOT and/or/not
df[df.region.isin(["West", "East"])]
df[df.region.str.contains("west", case=False, na=False)]
df[df.qty.between(3, 8)]
df[df.col.isna()]  /  df[df.col.notna()]
df.query("qty > 5 and region == 'West'")      # readable alternative
```

> **`.loc` is end-inclusive, `.iloc` is end-exclusive.** Guaranteed to appear in an assessment.

10. Pandas — Cleaning

```
df = df.rename(columns={"old": "new"})
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

df.drop(columns=["a", "b"])
df.drop_duplicates()                       # all columns
df.drop_duplicates(subset=["id"], keep="last")
df.duplicated(subset=["id"]).sum()

df.dropna()                                # any null in any column
df.dropna(subset=["qty"])
df.dropna(thresh=3)                        # keep rows with >=3 non-nulls
df.fillna(0)
df.fillna({"qty": 0, "region": "Unknown"})
df["qty"].fillna(df["qty"].median())
df["v"].ffill()  /  df["v"].bfill()        # forward / backward fill

df["qty"] = df["qty"].astype(int)
df["date"] = pd.to_datetime(df["date"], errors="coerce")   # bad -> NaT
df["amt"] = pd.to_numeric(df["amt"], errors="coerce")

df["region"] = df["region"].str.strip().str.title()
df["region"] = df["region"].replace({"W": "West"})
df["band"] = df["qty"].map({1: "low", 2: "mid"})
df["flag"] = np.where(df.qty > 5, "high", "low")
df["band"] = pd.cut(df.qty, bins=[0,3,6,99], labels=["low","mid","high"])
df["decile"] = pd.qcut(df.amt, 10, labels=False)           # equal-COUNT bins
```

`pd.cut` = equal-width bins you specify. `pd.qcut` = equal-frequency (quantile) bins. Know the difference.

11. Pandas — Aggregation

```
df.groupby("region")["revenue"].sum()
df.groupby(["region", "channel"])["revenue"].agg(["sum", "mean", "count"])

df.groupby("region").agg(
    revenue=("revenue", "sum"),
    orders=("order_id", "count"),
    avg_qty=("qty", "mean"),
    n_products=("sku", "nunique"),
).reset_index()                            # named aggregation — preferred

df.groupby("region")["revenue"].transform("sum")    # broadcasts back to row level
df.groupby("region").apply(lambda g: g.nlargest(3, "revenue"))
df.groupby("region").filter(lambda g: len(g) > 100)
```

`agg` reduces to one row per group; `transform` returns the original shape (use it to compute a share-of-group column); `filter` keeps or drops whole groups.

```
df.pivot_table(index="region", columns="quarter", values="revenue",
               aggfunc="sum", margins=True, fill_value=0)
pd.crosstab(df.region, df.channel, normalize="index")
df.melt(id_vars=["region"], var_name="metric", value_name="value")   # wide -> long
```

`pivot_table` is the direct pandas analogue of an Excel pivot — same mental model.

12. Pandas — Joining & Combining

```
pd.merge(left, right, on="key", how="inner")     # inner|left|right|outer|cross
pd.merge(l, r, left_on="id", right_on="ref", how="left")
pd.merge(l, r, on="key", suffixes=("_l", "_r"))
pd.merge(l, r, on="key", how="left", indicator=True)   # adds _merge column
l.join(r, on="key")                              # index-based
pd.concat([df1, df2], axis=0, ignore_index=True) # stack rows
pd.concat([df1, df2], axis=1)                    # side by side
```

Always check row counts before and after a merge. If the count grew, the right-hand key isn't unique and you have fan-out — every downstream sum is now inflated. `indicator=True` plus `df._merge.value_counts()` tells you exactly how many rows matched.

13. Pandas — Sorting, Ranking, Windows

```
df.sort_values("revenue", ascending=False)
df.sort_values(["region", "revenue"], ascending=[True, False])
df.nlargest(5, "revenue"); df.nsmallest(5, "revenue")

df["rank"] = df.groupby("region")["revenue"].rank(method="dense", ascending=False)
# method: average (default), min, max, first, dense

df["cum"]   = df.groupby("region")["revenue"].cumsum()
df["ma3"]   = df["revenue"].rolling(3).mean()
df["ma3c"]  = df["revenue"].rolling(3, center=True, min_periods=1).mean()
df["ewm"]   = df["revenue"].ewm(span=3).mean()
df["lag1"]  = df.groupby("sku")["revenue"].shift(1)
df["pct"]   = df["revenue"].pct_change()
df["expand"]= df["revenue"].expanding().mean()
```

14. Pandas — Dates

```
df["date"] = pd.to_datetime(df["date"])
df["year"]    = df.date.dt.year
df["month"]   = df.date.dt.month
df["quarter"] = df.date.dt.quarter
df["dow"]     = df.date.dt.dayofweek        # Monday = 0
df["dayname"] = df.date.dt.day_name()
df["week"]    = df.date.dt.isocalendar().week
df["month_start"] = df.date.dt.to_period("M").dt.to_timestamp()

df.set_index("date").resample("M")["revenue"].sum()     # M, W, Q, D, H
df.set_index("date").resample("W").agg({"revenue":"sum", "qty":"mean"})
pd.date_range("2025-01-01", periods=12, freq="MS")
df.date.max() - df.date.min()                           # Timedelta
```

`.dt` is the accessor for datetime methods on a Series, just as `.str` is for strings.

15. Output

```
df.to_csv("out.csv", index=False)
df.to_excel("out.xlsx", sheet_name="Summary", index=False)
with pd.ExcelWriter("multi.xlsx") as w:
    df1.to_excel(w, sheet_name="Raw", index=False)
    df2.to_excel(w, sheet_name="Summary", index=False)
df.to_parquet("out.parquet")
```

`index=False` is almost always what you want for CSV — otherwise you get a stray unnamed first column that causes trouble on re-read.

16. Gotchas Worth Memorising

SettingWithCopyWarning. Chained indexing may operate on a copy, so the assignment silently does nothing:

```
df[df.qty > 5]["flag"] = 1          # WRONG — may not stick
df.loc[df.qty > 5, "flag"] = 1      # RIGHT
sub = df[df.qty > 5].copy()         # explicit copy if you'll modify it
```

`inplace=True` is not faster and is being discouraged. Prefer reassignment: `df = df.dropna()`.

`&`, `|`, `~` for element-wise boolean ops — `and` / `or` / `not` raise “truth value of a Series is ambiguous”.

Parentheses are mandatory around each condition because `&` binds tighter than `>`.

`NaN != NaN`. Use `.isna()`, never `== np.nan`.

`axis=0` operates down rows (per column); `axis=1` operates across columns (per row). `df.mean()` defaults to `axis=0` — a per-column mean.

`apply` is a loop. Prefer vectorised operations, `np.where`, or `.map` for a dict lookup. `apply(axis=1)` in particular is very slow on large frames.

17. Practice Checklist

- [ ] Read a messy CSV with the right `dtype`, `parse_dates`, `na_values`
- [ ] Profile it: shape, dtypes, nulls per column, distinct counts
- [ ] Clean: trim/case-normalise strings, coerce types, handle nulls with a justified strategy
- [ ] Dedup on a business key, not on all columns
- [ ] Derive columns: date parts, a computed metric, a binned category

- [ ] `groupby().agg()` with named aggregations returning several metrics
- [ ] `pivot_table` cross-tab with margins
- [ ] Merge two frames and verify the row count is unchanged
- [ ] Rank within group, and compute share-of-group with `transform`
- [ ] Rolling mean and lag features on a time-ordered frame
- [ ] Resample daily data to monthly
- [ ] Write a multi-sheet Excel output